

Student name: Jordan Elkin

Student ID: 226621873

SIT225: Data Capture Technologies

Activity 1.1: Arduino Blink

Welcome to Arduino!

Arduino is an electronic prototyping platform. Different types of sensors & actuators can be attached to Arduino boards to create our own sensing-thinking-acting systems.

Throughout this unit, we will use Arduino to create different sensing devices, and to retrieve the collected sensor data.

In this task, we will try out an introductory exercise, to learn the basic concepts of Arduino.

Hardware Required

- Arduino Board with in-built LED

- USB cable

Software Required

- Arduino programming environment

Pre-requisites: You must do the following before this task

Unit site weekly materials

Active learning sessions require reading material and/or videos available in the unit site, which you must read/view **BEFORE** you start the lab. If you are on-campus, this means that we expect you to have gone through these materials when you join active learning sessions.

Why should you read/watch pre-lab materials?

These materials will help you understand the background which the lab tasks require. Students come to university from diverse backgrounds. Some of you may be familiar with the background information, some of you may not. When you come to the lab prepared, you're already equipped

with confidence and will be able to participate in activities better. Ultimately, class time will be much more productive, dynamic, and fun for everyone.

Here are the pre-lab materials for our first task:

1. Watch TED Talk: <https://www.youtube.com/watch?v=UoBUXOOdLXY> (~15 minutes)
2. Read Arduino tutorial: <https://www.dummies.com/article/technology/computers/hardware/arduino/how-to-complete-your-first-arduino-sketch-164747>)
3. Read this task sheet from beginning to end.

Task Objective

“We have an Arduino board with an in-built LED light. We need the LED light to be turned on and off continuously, every one second.”

Activity Submission Details

Answer the questions below in this word document and other activities in this activity sheet to create a PDF and submit to OnTrack as described in this week's OnTrack task. PDFs of this activity sheet and OnTrack task need to be merged for submission in the OnTrack portal.

Q1: The TED talk given under the Pre-Lab materials, shows how Arduino is being used for interesting projects to capture data from the environment, process it, and use it to carry out useful actions.

Fill the given table below to answer the following:

What are **three** projects that use captured data as given in the TED talk? What data do they capture? What sensors do you think they could use to capture this data?

Project name	Data captured	Sensors to capture the data
Pet Feeder	Recognition of different cats, proximity of cat to feeder.	RFID reader that identifies which cat has approached the feeder, Proximity sensors sense when cat is still within the pet feeder itself.
NYU/ITP (PS3 Controller)	Head movement, button click	IR webcam captures the position of the reflector on a baseball cap which then moves the cursor on the laptop in the same direction as the tracked movement. A mechanical button captures button clicks and when combined with the onscreen overlay allows for the pressing of on screen buttons with the mechanical button.

Sign Language glove	Hand flexion (i.e. finger position), x,y,z axis measurements	Flex sensor to determine the position of the fingers and workout the sign that is being made by the user which is then translated into text or speech. The accelerometer is detecting the position of the hand and its placement
---------------------	--	--

Q2: Consider the given Task Objective. Think about how this simple system can be decomposed to 'Sense-Think-Act'?

- a) What is the 'sensing' requirement in this system, if any?

There is a sensing requirement for this system because the task requires the light to be turned on and off and this should be verified. Since the LED light is an output device that turns on and off when a pin is set high (i.e. the voltage on that pin is 3.3v or 5v) or the pin is set low respectively, and to know that the LED light is indeed on we could add an additional sensing pin so that we can determine if the light has been turned on successfully or not. Therefore verifying the requirement that the LED light must turn on and off continuously every second and also sensing the state of the LED light.

- b) What is the 'thinking' requirement in this system, if any?

There is a thinking requirement in that we could record the amount of times that the pin is set high or low and analyse the data. But for this specific task the primary thinking requirement is to determine the state of the LED light from the information provided by a sensor which in this case is a sensing pin which will have a voltage when the light is on and no voltage when its off.

- c) What is the 'acting' requirement in this system, if any?

The primary acting requirement for this system is for the LED light to be turned on when it is off and off when it is on continuously every second.

Q3: Please refer to the provided 'Arduino Blink Activity Sheet' and follow the steps.

- a) In Arduino-speak, what is a "sketch"?

A sketch is the source code file for a program which runs on an Arduino. A sketch is usually written in C/C++ and is the way to write software that will run on the Arduino and interact with sensors and other devices with the Arduino.

- b) `setup()` and `loop()` are key Arduino constructs. These are required in every Arduino sketch.

- i) Which of the above two, runs once at the very beginning of your program and never again (unless you reset or upload new code)?

The `setup()` function runs once at the beginning of the program and sets up and initialises objects that are needed at runtime such as the Serial object/

- ii) Which of the above two, is used to continuously run code over and over again?

The `loop()` function runs continuously and causes the code written within to be executed forever until a new program is loaded into memory or the device is shutdown. The `loop()` function is run immediately after the `setup()` function has been executed and ended.

- c) What does **`pinMode()`** do?

The `pinMode()` function allows the developer to set the specified pins as an `INPUT`, `OUTPUT`, or `INPUT_PULLUP`. These specific modes allow for the pin to be set to high or low for output meaning that a voltage is now at the pin which allows for the actuating of led lights or a signal to be sent to a sensor connected at the pin. Whereas, the input mode allows for the reading of a pin which means monitoring for changes such as High → low or low → high. This also enables sensors to send data to the pin by setting the input pin as either low or high (0 or 1) and sending the data bit by bit.

Hint: <http://arduino.cc/en/Reference/HomePage>

- d) What is a comment?

A comment which can be multi-line or single line `/**/` or `//` respectively, allows the developer to add additional explanation for code function when it is not clear why the code is written a certain way or its not clear as to why certain constants are set the way that they are.

- e) What does the following line of code do:

```
delay(x);
```

This method causes the Arduino to sleep or wait for a specific amount of seconds which in the above example is the x argument.

Hint: <http://arduino.cc/en/Reference/HomePage>

- f) There is something you need to check before uploading your sketch. What is this?

Q4: How can you test the Blink program to make sure it is working as given in the Task Objective?

Firstly, you can run the verify option in the IDE. This will check for any errors that may cause the program to not function as required in the task objective. Secondly, you could alter the blink program and attach a new LED light to a pin and then set the pin high and low every second which would verify that the Blink program is functioning as intended. Thirdly, you can check where the inbuilt LED is on the Arduino and time it and check that it is indeed turning off and on continuously every second. Lastly, you could have the Arduino write to the serial port every second that it turns the light on and off and with these debug messages you could cross reference with the physical device and check that the light is matching up with the serial messages.

Activity 1.2: Write Arduino data to serial communication port

Now you can blink Arduino's built-in LED, it is time to talk to outside Arduino-world, your computer which connects the Arduino board using a USB cable. Arduino IDE shows what you write to the serial port.

Hardware Required

Arduino Board with in-built LED

USB cable

Software Required

Arduino programming environment

Steps:

Steps	Actions
1	Identify the port through which Arduino is connected to your computer. You can find it in Arduino IDE Tools menu. Write Arduino sketch (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/sketch_blink.ino), which look like below, in Arduino IDE, deploy in your board and observe the output in IDE's serial monitor.

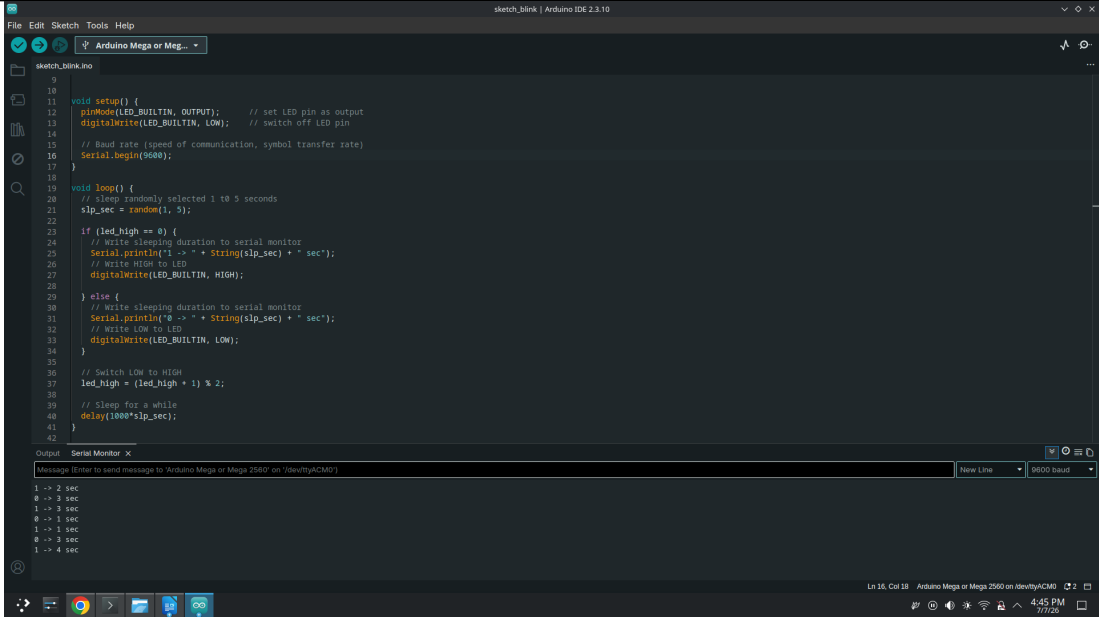
```

1  int led_high = 0; // variable to flag ON (1) or OFF (0)
2  int slp_sec = 0; // variable holds number of seconds to sleep
3
4
5  void setup() {
6      pinMode(LED_BUILTIN, OUTPUT); // set LED pin as output
7      digitalWrite(LED_BUILTIN, LOW); // switch off LED pin
8
9      // Baud rate (speed of communication, symbol transfer rate)
10     Serial.begin(6900);
11 }
12
13 void loop() {
14     // sleep randomly selected 1 to 5 seconds
15     slp_sec = random(1, 5);
16
17     if (led_high == 0) {
18         // Write sleeping duration to serial monitor
19         Serial.println("1 -> " + String(slp_sec) + " sec");
20         // Write HIGH to LED
21         digitalWrite(LED_BUILTIN, HIGH);
22     } else {
23         // Write sleeping duration to serial monitor
24         Serial.println("0 -> " + String(slp_sec) + " sec");
25         // Write LOW to LED
26         digitalWrite(LED_BUILTIN, LOW);
27     }
28
29     // Switch LOW to HIGH
30     led_high = (led_high + 1) % 2;
31
32     // Sleep for a while
33     delay(1000*slp_sec);
34 }
35
36

```

Question: Screenshot the serial monitor output and paste the image here.

Answer: Please see image below

	
2	Question: Observe the use of “Serial” such as functions Serial.begin() in setup and Serial.println() in loop. Describe what these functions are doing with respect to the serial monitor output you have attached above. Answer: The Serial being method initialises the serial monitor connection and establishes the baud rate which in the screenshot above I set to 9600. The Serial.println() method in the loop above writes to the serial monitor a string which has been passed to the method and is then displayed on the serial monitor console in the Arduino IDE.
3	Question: If Arduino transfers data at 4800 bits per second (baud rate) and you're sending 12 bytes of data, how long does it take to send over this information? Answer: If the baud rate is 4800 bits per second and 12 bytes of data is sent then it will take approximately 0.02 seconds (20 ms) for the data to be sent. This is calculated as shown below. The calculation is only an approximation as it does not account for any padding bits or parity bits. baud rate = 4800 data = 12 bytes => 96 bits transfer speed = 4800 bits / second => 4.8 bits / ms 96 / 4.8 = 20 ms

Activity 1.3: Arduino talks to Python

To listen to what Arduino sends, there will be a Python program running and keep listening to the same port where Arduino is writing data to receive it.

Hardware Required

Arduino Board with in-built LED

USB cable

Software Required

Arduino programming environment

Python 3.0 (Follow Python installation manual in unit site)

Steps:

Steps	Actions
1	Write Arduino sketch (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/sketch_serial_comm.ino) which looks like below. Open in Arduino IDE. Study the code. Upload the code to Arduino board and observe output in Arduino IDE serial monitor.

```

1
2 int x;
3
4 void setup() {
5     Serial.begin(9600); // set baud rate
6 }
7
8 void loop() {
9     while (!Serial.available()) {} // wait for data to arrive
10
11     // read string data from Serial, we know Python
12     // script is sending just an integer.
13     x = Serial.readString().toInt();
14
15     // write a string (no newline)
16     Serial.print("Arduino sends: ");
17
18     // Add 1 to what received.
19     // Write an integer with a newline (println vs print).
20     Serial.println(x + 1);
21
22     // Push the data through serial channel.
23     Serial.flush();
24 }

```

Question: Do you see any output in serial monitor? If not, then why?

Answer: Yes you see output in the serial monitor when you send information over the serial connection to the arduino. The arduino is currently only set to expect an integer value which it will then add 1 to and then write to the serial monitor through the serial objects print method. However, merely opening the serial monitor would not cause any output to appear.

- 2 Write Python code as below (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/serial_comm_script.py) and save it to a file serial_comm_script.py.

```

1 import serial
2 import random
3
4 # set baud rate, same speed as set in your Arduino sketch.
5 boud_rate = 9600
6
7 # set serial port as suits your operating system
8 s = serial.Serial('/dev/ttyACM1', boud_rate, timeout=5)
9
10 while True: # infinite loop, keep running
11
12     # a random number between 5 and 50.
13     data_send = random.randint(5, 50)
14
15     # write to serial port, set data encoding.
16     # Raw bytes are sent through serial ports, Python bytes() needs
17     # to know the encoding to generate bytes from string.
18     #
19     # We send a single integer which is read from Arduino sketch.
20     #
21     d = s.write(bytes(str(data_send), 'utf-8'))
22     print(f"Send >>> {data_send} ({d} bytes)")
23
24     # Read from serial port.
25     #
26     # readline keeps reading until a newline found in the data stream.
27     # Unlike write above, we just send an integer with no newline.
28     # You should receive data the same way as it is sent.
29     #
30     d = s.readline().decode("utf-8")
31     print(f"Recv <<< {d}")
32

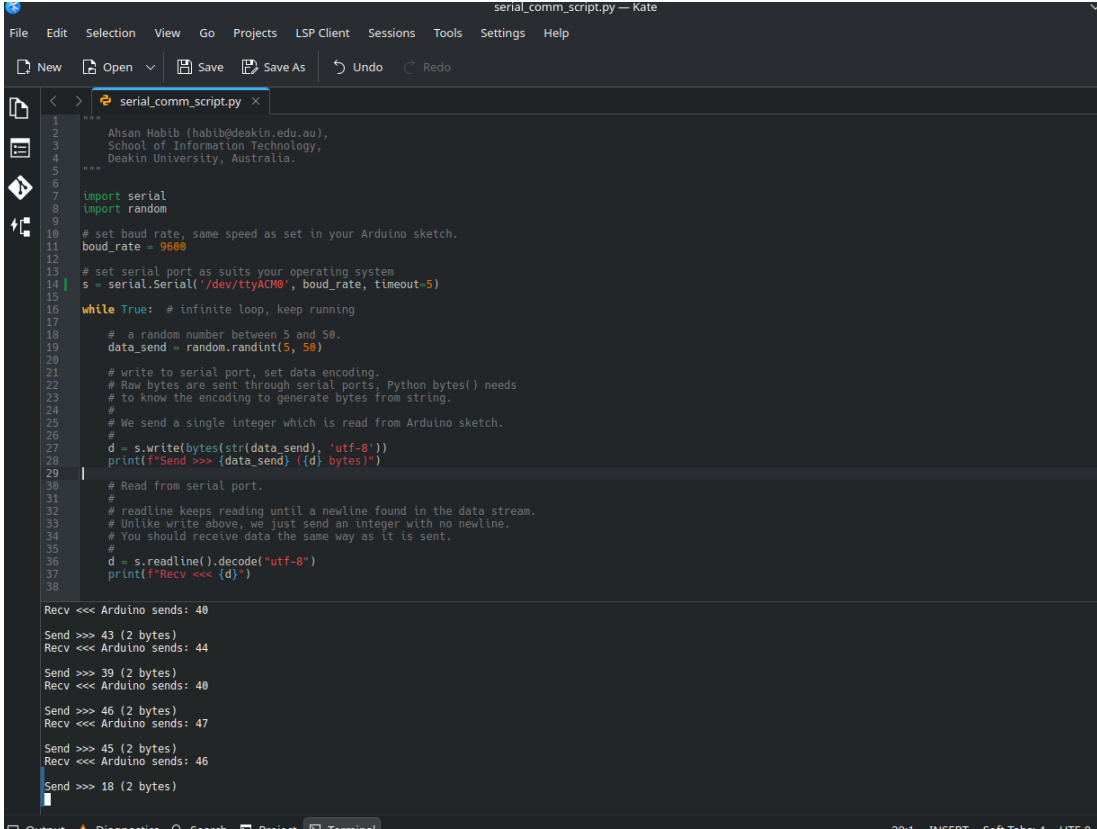
```

Run the Python file from command line using command:

\$ *python serial_comm_script.py*

Question: Run Python script in command line, does the script run or do you receive any error? If there is an error, analyse the error message and identify what went wrong.

Answer: initially the script did not run because the serial port on my computer was a different number than the one hard-coded in the script. Further, since I had the arduous IDE running with its serial monitor opened at the same time the resource was busy so no serial communications could start. However, after making the necessary changes to the script and closing the Arduino IDE I was able to connect to the Arduino over the serial interface and the program was able to send and receive the data no problem.

3	Question: Following the above step 2, do you think Arduino IDE is keeping the serial communication port busy talking to Arduino board? Now try to close the Arduino IDE and repeat step 2 above to run the Python script. Do previous errors show up again? If not, Python script should have print messages in the command line. Take the screenshot of the Python script output and paste here. Answer: Yes the Arduino IDE is keeping the port busy when its serial monitor is opened. As previously stated these issues were corrected once the serial monitor is closed or the IDE is closed down completely, either option results in the serial interface being opened.  <pre> 1 """ 2 Ahsan Habib (habib@deakin.edu.au), 3 School of Information Technology, 4 Deakin University, Australia. 5 """ 6 7 import serial 8 import random 9 10 # set baud rate, same speed as set in your Arduino sketch. 11 baud_rate = 9600 12 13 # set serial port as suits your operating system 14 s = serial.Serial('/dev/ttyACM0', baud_rate, timeout=5) 15 16 while True: # infinite loop, keep running 17 18 # a random number between 5 and 50. 19 data_send = random.randint(5, 50) 20 21 # write to serial port, set data encoding. 22 # Raw bytes are sent through serial ports; Python bytes() needs 23 # to know the encoding to generate bytes from string. 24 # 25 # We send a single integer which is read from Arduino sketch. 26 # 27 d = s.write(bytes(str(data_send), 'utf-8')) 28 print(f"Send >>> {data_send} ({d} bytes)") 29 30 # Read from serial port. 31 # 32 # readline keeps reading until a newline found in the data stream. 33 # Unlike write above, we just send an integer with no newline. 34 # You should receive data the same way as it is sent. 35 # 36 d = s.readline().decode("utf-8") 37 print(f"Recv <<< {d}") 38 39 Recv <<< Arduino sends: 40 40 Send >>> 43 (2 bytes) 41 Recv <<< Arduino sends: 44 42 Send >>> 39 (2 bytes) 43 Recv <<< Arduino sends: 40 44 Send >>> 46 (2 bytes) 45 Recv <<< Arduino sends: 47 46 Send >>> 45 (2 bytes) 47 Recv <<< Arduino sends: 46 48 Send >>> 18 (2 bytes) </pre>
4	Question: Observe the Python script output and describe the communication protocol used between Arduino sketch and Python script. Answer: The communication protocol that is occurring is known as the serial communication. There are multiple types of serial communications these include SPI, I2C and UART. By default the serial interface for the Arduino is the Universal asynchronous receiver-transmitter (UART). Where one pin is used for transmission and the other is for receiving on. In the script the Arduino is sent a string in the form of raw bytes which is a utf-8 encoding. This means that characters are encoded in a

	series of 8 bits for ASCII characters and up to 4 bytes for other languages. These bits are then transmitted to the Arduino one bit at a time and then the inverse is conducted when the Arduino writes to the serial interface and the python program reads the bytes it receives from the Arduino stopping only when a newline character is encountered which triggers the contents being printed to the console.
--	--